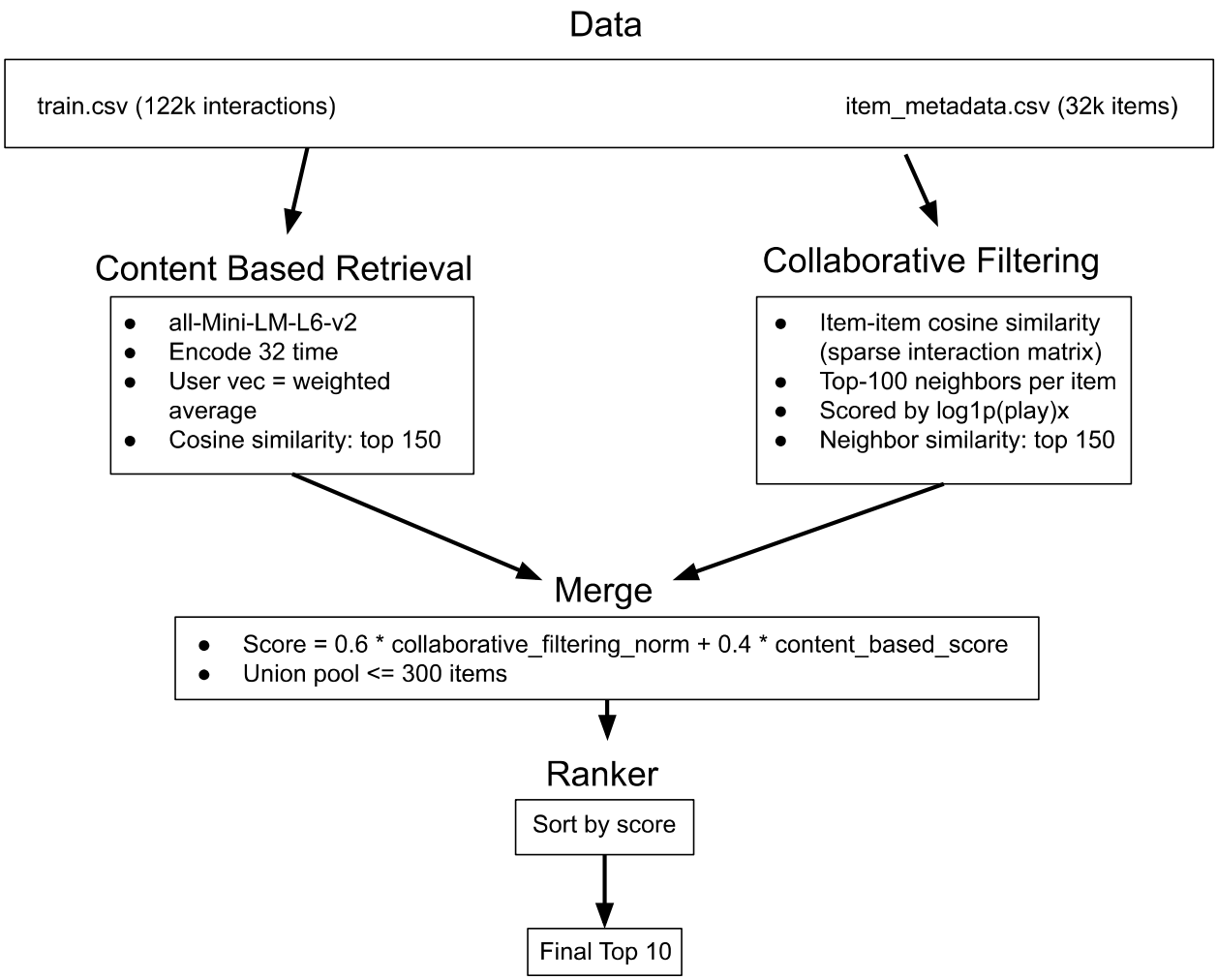
1. Architecture Diagram



2. Retrieval Recall

K - Value	**Content Based**	**Collaborative Filtering**	**Hybrid**
100	0.0778	0.1328	0.1250
150	0.1004	0.1586	0.1472

200	0.1004	0.1586	0.1750

3. Ranking Features

Feature	**Group**	**Description**
hist_len	user	Num training interactions
user_vec	user	Weighted average of embeddings over training history
warm_flag	item	If item appears in training data
item_embedding	item	all-MiniLM-L6-v2 encoding of an item's data
cb_score	interaction	Cosine similarity
cf_score_norm	interaction	Weighted sum of item to item interactions over history
hybrid_score	interaction	0.6 * sf_score_norm + 0.4 * sb_score. Final ranking signal

Note: 0.6 and 0.4 were chosen after trying several different combinations that were found not as accurate.

4. Cold-user / cold-item handling

What happens varies depending on what route is taken.

1. Content Based pipeline:

    a. Builds a user vector from the available item embeddings, even if small.
    b. This pipeline still always returns enough candidates such that the rest of the program isn't disturbed.
2. Collaborative filtering:
    a. For each of the items available, sum over their top 100 neighbors.
    b. Fills up enough items for downstream tasks.
3. Hybrid merge
    a. Fill in the gap from whatever pipeline is lacking
4. Popularity
    a. A fallback if the final merged pool has less than 10 unique un-seen items.
5. Cold items
    a. Collaborative filtering is less effective in this case.
    b. Content based is better since it does embed similarity.
    c. Hybrid then makes up for this lack of collaborative filtering contribution.